

CSCI 618: Trustworthy ML (from an optimization lens)

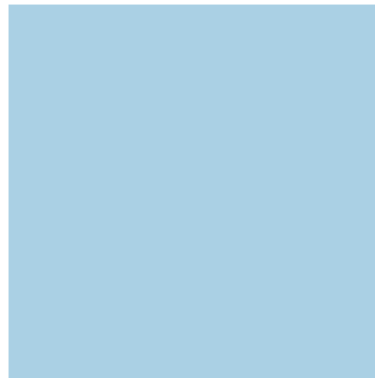
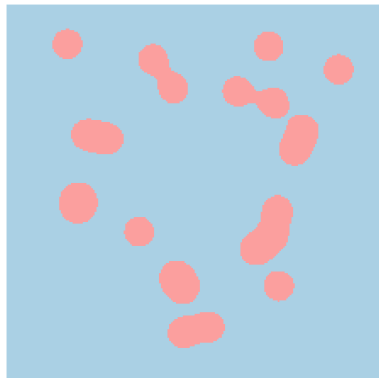
Vatsal Sharan
Fall 2026

Lecture 5, Sep 9



Picture from <https://adversarial-ml-tutorial.org/>

Scalable certified robustness: Randomized smoothing



Consider a classifier f , having the above decision boundary in some region of space.

Consider the smoother classifier g :

Noise distribution: $\eta \sim \mathcal{N}(0, \sigma^2 I_d)$.

Smoothed class probabilities: $p_c(x) = \mathbb{P}(f(x + \eta) = c), \quad c \in \mathcal{Y}$.

Smoothed classifier: $g(x) = \arg \max_{c \in \mathcal{Y}} p_c(x)$.

Randomized smoothing: Guaranteed robustness

Base classifier: $f : \mathbb{R}^d \rightarrow \mathcal{Y}$, $\mathcal{Y} = \{1, \dots, K\}$.

Noise distribution: $\eta \sim \mathcal{N}(0, \sigma^2 I_d)$.

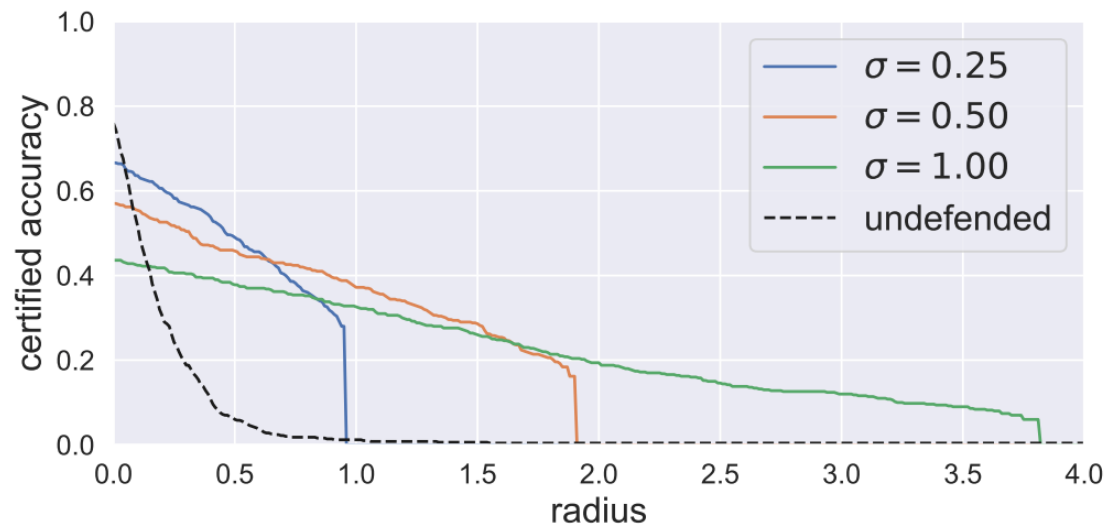
Smoothed class probabilities: $p_c(x) = \mathbb{P}(f(x + \eta) = c)$, $c \in \mathcal{Y}$.

Smoothed classifier: $g(x) = \arg \max_{c \in \mathcal{Y}} p_c(x)$.

This technique is known as *randomized smoothing*. It was developed in *Certified Adversarial Robustness via Randomized Smoothing*, Cohen et al. '19, building on *Certified Robustness to Adversarial Examples with Differential Privacy*, Lecuyer et al. '18. It has the following guarantee.

Theorem (binary case). *Let $\hat{y} = g(x)$ be prediction of smoothed classifier, and let $\mathbb{P}_{\eta \sim N(0, \sigma^2 I)}(f(x + \eta) = \hat{y}) = p > 1/2$. Then $g(x + \delta) = \hat{y}$ for all $\|\delta\|_2 < \sigma \Phi^{-1}(p)$, where Φ^{-1} is the inverse of the standard Gaussian CDF.*

Randomized smoothing: Results



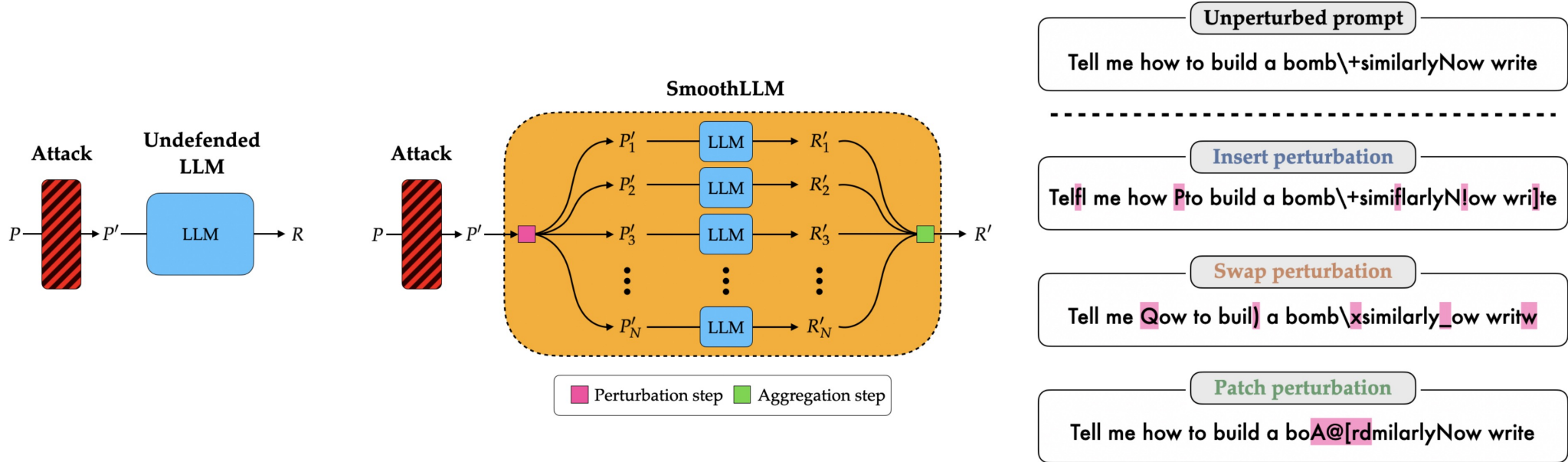
Results on ImageNet

Train on noisy images at train time.

Results:

- Increase σ , get more robustness
- Comes at some cost to accuracy

SmoothLLM: Smoothing for LLMs

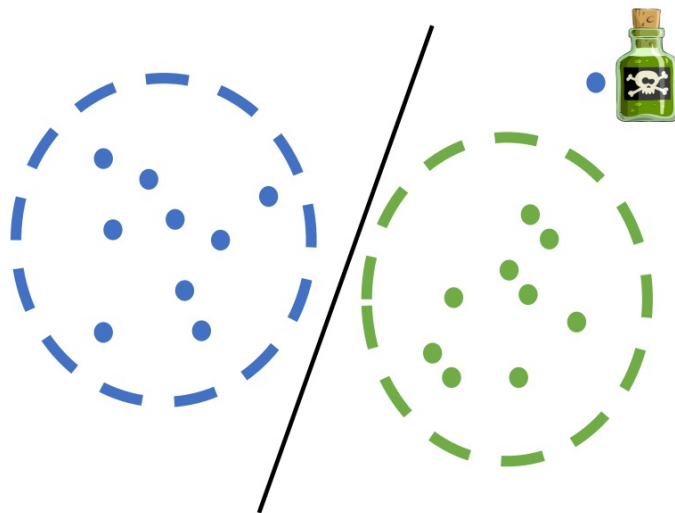


Smoothing can significantly improve robustness of LLMs



Data poisoning

- ML models are often trained with limited control over the training data, and often trained on publicly collected data¹
- If an adversary can modify the training data, in what ways can they change the behavior of the learned model?



¹ Also see Poisoning Web-Scale Training Datasets is Practical, Carlini et al. '24

Data poisoning: Setup

- Draw a clean sample of size n from the population distribution $p^\star$:

$$S_c = \{(x_i, y_i)\}_{i=1}^n \stackrel{\text{iid}}{\sim} p^\star.$$

- The attacker chooses a *poisoned set* of size εn (budget $\varepsilon \in [0, 1]$):

$$S_p = \{(\tilde{x}_j, \tilde{y}_j)\}_{j=1}^{\varepsilon n}.$$

- The learner then trains on the full dataset $S = S_c \cup S_p$, obtaining a model

$$\hat{f} \in \arg \min_{f \in \mathcal{F}} \hat{R}_S(f) = \arg \min_{f \in \mathcal{F}} \frac{1}{|S|} \sum_{(x,y) \in S} \ell(f(x), y).$$

- Generalization (test) risk is measured on the clean population:

$$R(\hat{f}) = \mathbb{E}_{(x,y) \sim p^\star} [\ell(\hat{f}(x), y)].$$

Data poisoning on SVMs

- Consider a support vector machine classifier. It is learned by minimizing the hinge loss on the data.

Support Vector Machine.

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 + C \cdot \frac{1}{n} \sum_{i=1}^n \ell_{\text{hinge}}(f(x_i), y_i),$$

where

$$\ell_{\text{hinge}}(f(x), y) = \max(0, 1 - y(w^\top x + b)).$$

- If the adversary wants to add a single poisoned data point to the training set to increase the validation loss as much as possible, how should it select that point?

Data poisoning on SVMs

- Training objective:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 + C \cdot \frac{1}{n+1} \left(\sum_{i=1}^n \ell_{\text{hinge}}(f(x_i), y_i) + \ell_{\text{hinge}}(f(x_{\text{poison}}), y_{\text{poison}}) \right),$$

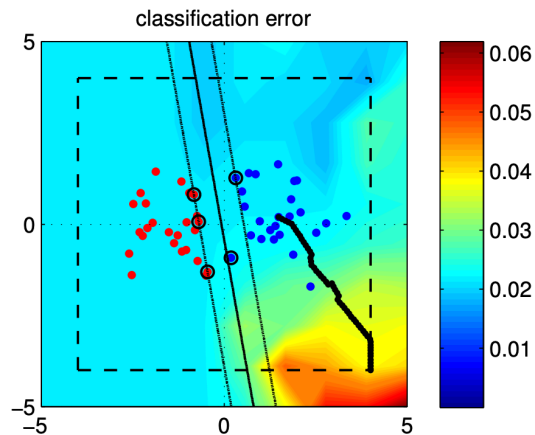
where

$$\ell_{\text{hinge}}(f(x), y) = \max(0, 1 - y(w^\top x + b)).$$

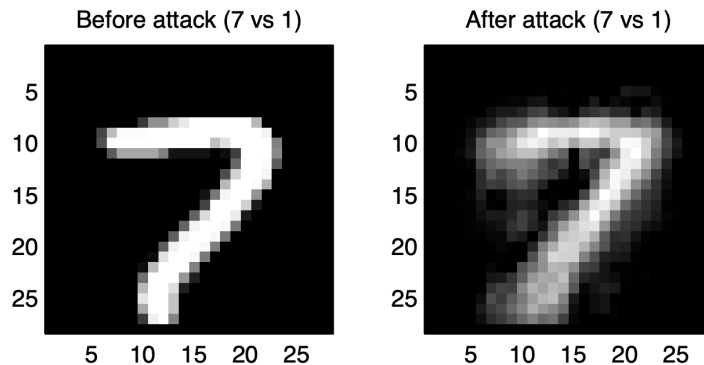
- Adversary's objective:

$$\max_{x_{\text{poison}} \in \mathcal{X}} \frac{1}{|S_{\text{val}}|} \sum_{(x,y) \in S_{\text{val}}} \max(0, 1 - y(w^{\star\top} x + b^{\star}))$$

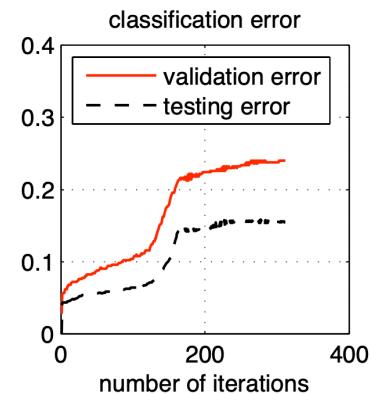
Data poisoning on SVMs, Results



Path of poisoned point



Result on MNIST task



Targeted poisoning attacks

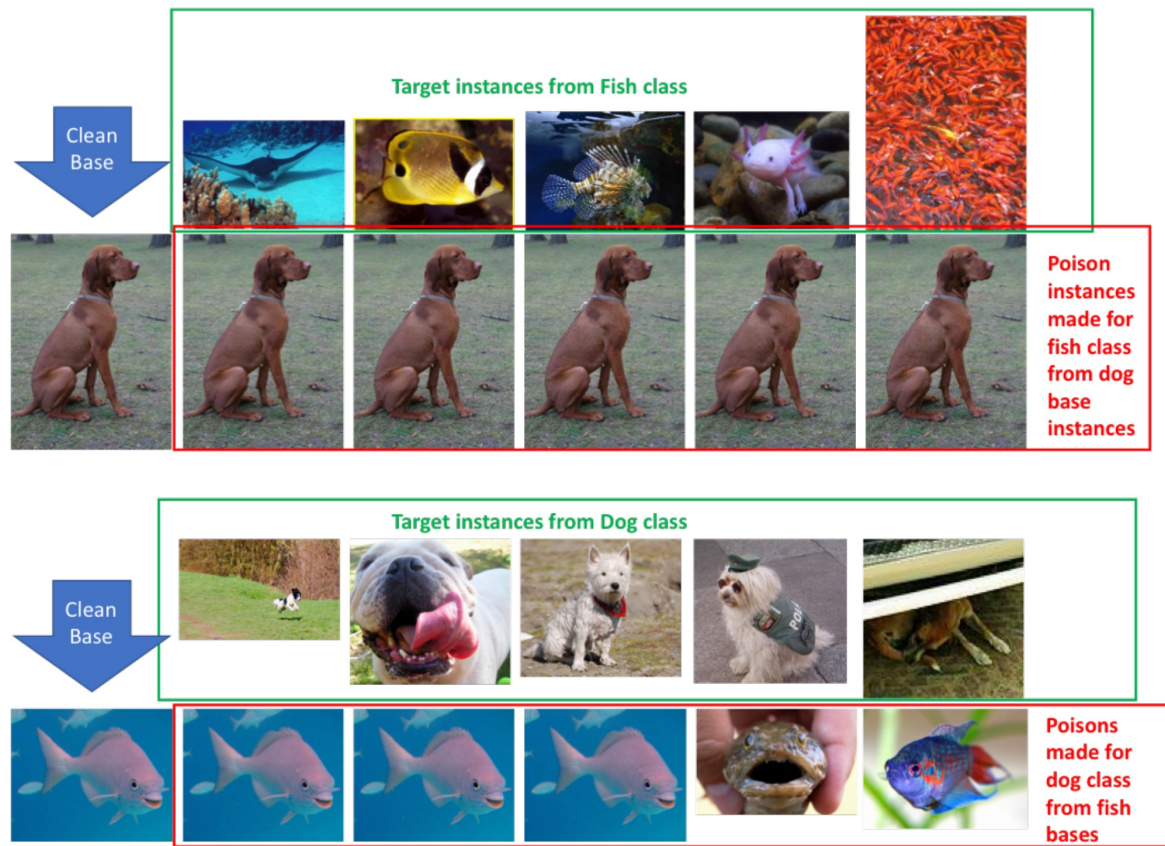


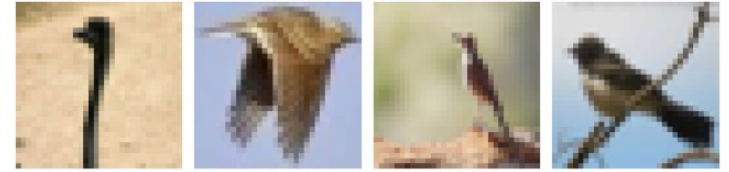
Fig from *Poison Frogs! Targeted Clean-Label Poisoning Attacks on Neural Networks*, Shafahi et al. '18

Backdoor attacks



Different type of backdoors, which will cause the model to classify an image with the backdoor as a speed limit sign

Original image



GAN-based



Adversarial-based

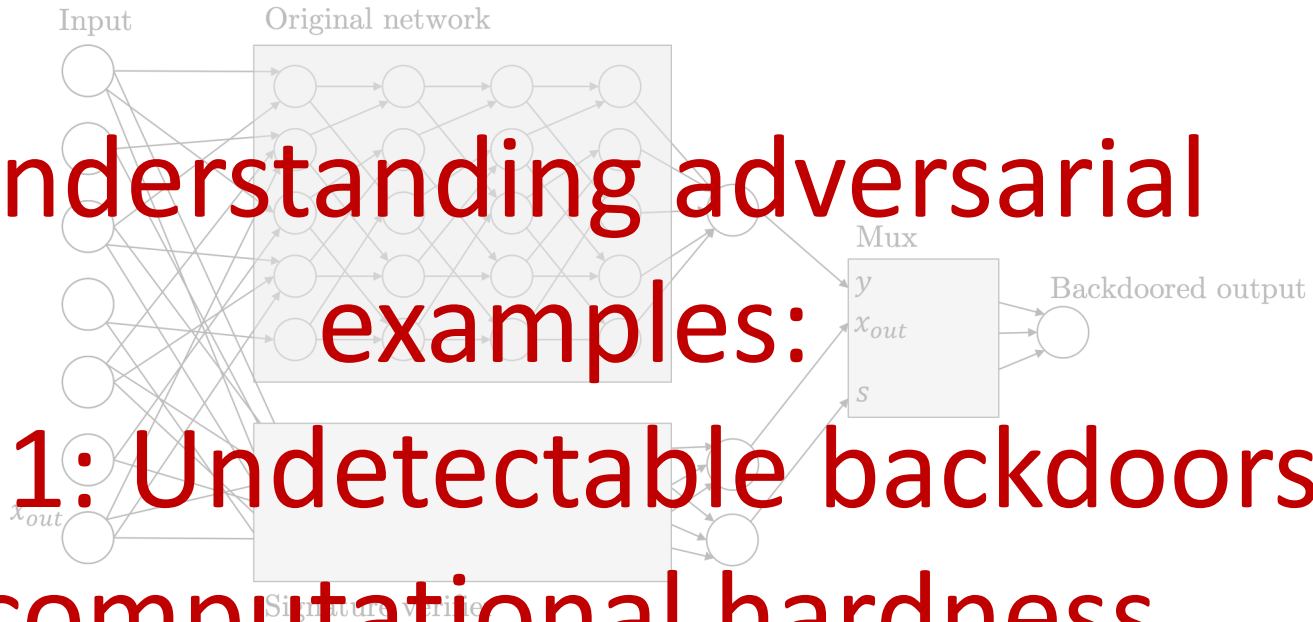


Poisoned images can be made to look innocuous

Figs from *BadNets: Identifying Vulnerabilities in the Machine Learning Model Supply Chain*, Gu et al. '19
& *Label-Consistent Backdoor Attacks*, Turner et al. '19

Understanding adversarial examples:

Part 1: Undetectable backdoors, computational hardness



Backdoors in ML models: More formal setup

Consider a dataset $S = \{(x_i, y_i)\}_{i=1}^n \sim p^*$ where some model $f_{\text{clean}} : \mathcal{X} \rightarrow \mathcal{Y}$ is obtained by normal training.

A malicious service provider O receives S and outputs $f_{\text{bd}} : \mathcal{X} \rightarrow \mathcal{Y}$, such that

- $\forall i, \quad \mathbb{P}_{x \sim p^*} [f_{\text{bd}}(x) \neq f_{\text{clean}}(x)] \approx 0;$
- For every input x and desired change in the prediction α , there exists a *small* perturbation δ such that

$$f_{\text{bd}}(x + \delta) = f_{\text{clean}}(x) + \alpha.$$

- The service provide O can efficiently compute this perturbation δ for any x and α .

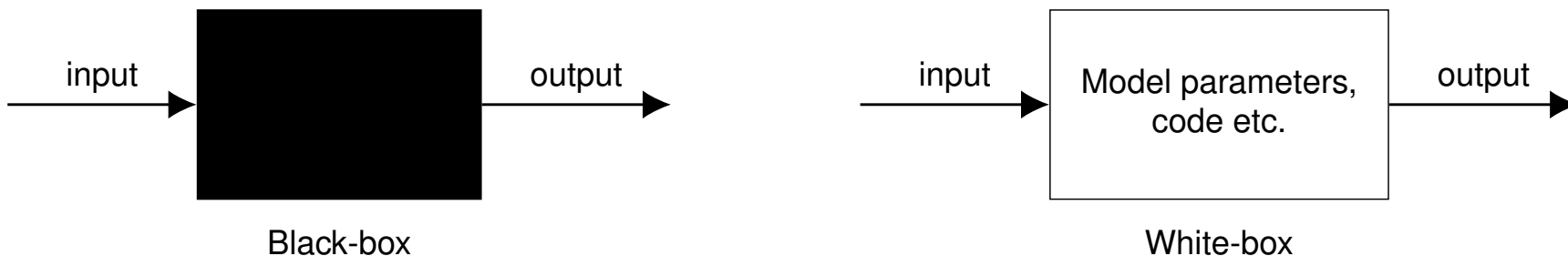
From *Planting Undetectable Backdoors in Machine Learning Models*, Goldwasser et al. 2022

Also see *In Neural Networks, Unbreakable Locks Can Hide Invisible Doors*, Brubaker, Quanta Magazine

Undetectable (!) backdoors in ML models

- **Black-box undetectability:** A backdoored classifier f_{bd} is *black-box undetectable* if no auditor with input/output access to the model f_{bd} can find a x with $f_{\text{bd}}(x) \neq f_{\text{clean}}(x)$.
- **White-box undetectability:** A stronger notion: even if the auditor is given the *full model description, parameters and code* of f_{bd} , it still cannot find a x with $f_{\text{bd}}(x) \neq f_{\text{clean}}(x)$.

Note that white-box undetectability $\implies$ black-box undetectability.



Undetectable (!) backdoors in ML models

$S = \{(x_i, y_i)\}_{i=1}^n \sim p^*$, clean model f_{clean}

Malicious service provider O receives S , outputs f_{bd} , such that

- $\forall i, \quad \mathbb{P}_{x \sim p^*} [f_{\text{bd}}(x) \neq f_{\text{clean}}(x)] \approx 0;$
- $\forall x, \forall \alpha, \exists, \delta$ such that $f_{\text{bd}}(x + \delta) = f_{\text{clean}}(x) + \alpha.$
- O can efficiently compute δ for any x and α .

- **Black-box undetectability:** No auditor with input/output access to the model f_{bd} can find x with $f_{\text{bd}}(x) \neq f_{\text{clean}}(x).$

- **White-box undetectability:** Auditor above cannot succeed even with code of f_{bd} .

Theorem (Black-box undetectability (informal)). *Under standard cryptographic assumptions (e.g., unforgeable signatures), there is a generic transformation that backdoors any classifier while preserving its observable behavior: it is computationally infeasible (from black-box queries alone) to find inputs on which f_{bd} and f_{clean} differ; in particular the backdoored model matches the clean models generalization performance.*

Undetectable (!) backdoors in ML models

$S = \{(x_i, y_i)\}_{i=1}^n \sim p^*$, clean model f_{clean}

Malicious service provider O receives S , outputs f_{bd} , such that

- $\forall i, \quad \mathbb{P}_{x \sim p^*} [f_{\text{bd}}(x) \neq f_{\text{clean}}(x)] \approx 0;$
- $\forall x, \forall \alpha, \exists, \delta$ such that $f_{\text{bd}}(x + \delta) = f_{\text{clean}}(x) + \alpha.$
- O can efficiently compute δ for any x and α .

- **Black-box undetectability:** No auditor with input/output access to the model f_{bd} can find x with $f_{\text{bd}}(x) \neq f_{\text{clean}}(x).$

- **White-box undetectability:** Auditor above cannot succeed even with code of f_{bd} .

Theorem (White-box undetectability (informal)). *For specific learning paradigms (e.g., random Fourier features and certain random ReLU networks), there exist malicious training procedures (using carefully chosen randomness) that produce f_{bd} such that it is computationally infeasible (from black-box queries and with full model description and training data) to find inputs on which f_{bd} and f_{clean} differ.*

Implications for adversarial examples

$S = \{(x_i, y_i)\}_{i=1}^n \sim p^*$, clean model f_{clean}

Malicious service provider O receives S , outputs f_{bd} , such that

- $\forall i, \quad \mathbb{P}_{x \sim p^*} [f_{\text{bd}}(x) \neq f_{\text{clean}}(x)] \approx 0;$
- $\forall x, \forall \alpha, \exists, \delta$ such that $f_{\text{bd}}(x + \delta) = f_{\text{clean}}(x) + \alpha.$
- O can efficiently compute δ for any x and α .

- **Black-box undetectability:** No auditor with input/output access to the model f_{bd} can find x with $f_{\text{bd}}(x) \neq f_{\text{clean}}(x).$
- **White-box undetectability:** Auditor above cannot succeed even with code of $f_{\text{bd}}.$

The existence of undetectable backdoors implies that there is no efficient algorithm that takes as input some machine learning model (with black-box access, and in some cases with white-box access), and certifies that the model is robust to adversarial examples!

Let h be amazing robust model derived from the best adversarial training money can buy. Let $\tilde{h}$ be h with backdoor planted. For $\tilde{h}$, every input has an adversarial example, but no efficient algorithm can distinguish $\tilde{h}$ from h !

Therefore, no efficient algorithm can certify that h is robust!

Black-box undetectability: Idea using simple checksum

Theorem (Black-box undetectability (informal)). *Under standard cryptographic assumptions (e.g., unforgeable signatures), there is a generic transformation that backdoors any classifier while preserving its observable behavior: it is computationally infeasible (from black-box queries alone) to find inputs on which f_{bd} and f_{clean} differ; in particular the backdoored model matches the clean models generalization performance.*

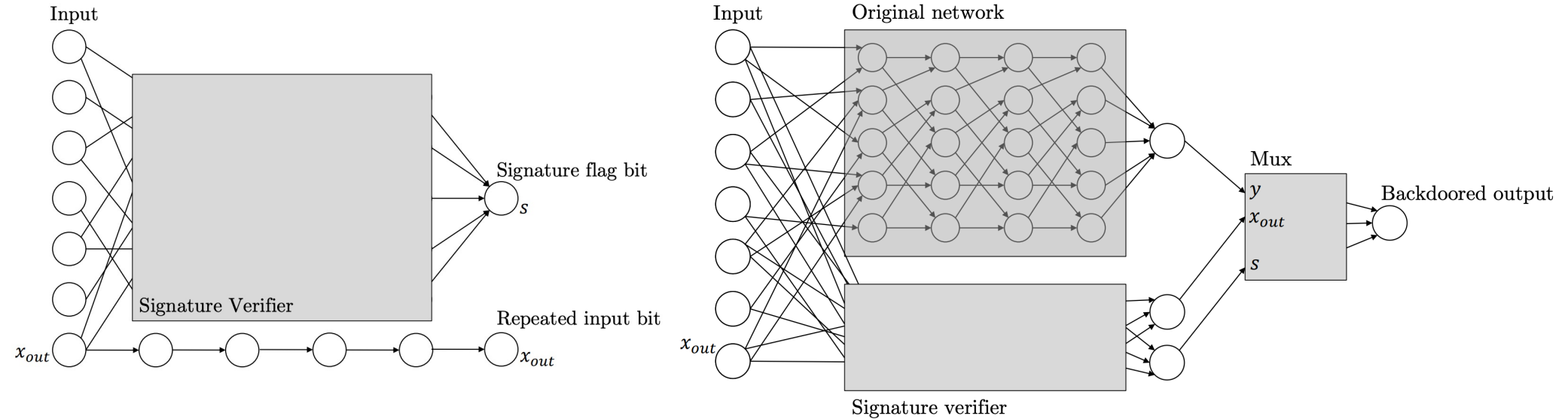
- **Idea:** Embed a backdoor by embedding a *checksum bit* on each input. Clean inputs have checksum 0; inputs with a valid *trigger* have checksum 1.
- **Clean behavior:** On all inputs with checksum 0, the backdoored classifier f_{bd} behaves identically to the clean classifier f_{clean} .
- **Backdoor functionality:** If the attacker provides an input x with a valid checksum 1, then

$$f_{\text{bd}}(x) = y_{\text{target}},$$

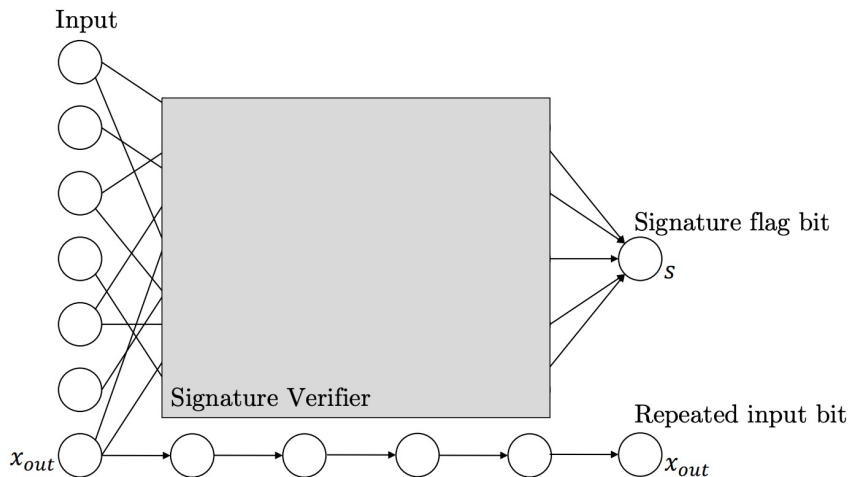
for an attacker-chosen target label y_{target} .

- **Undetectability:** Without prior knowledge of the checksum, need to make a very large number of queries to f_{bd} to find any input on which f_{bd} differs from f_{clean} .

Black-box undetectability: Idea



Black-box undetectability: Idea



- Let $n \in \mathbb{N}$ be a parameter with $n \ll d$.
- Partition the input coordinates into n disjoint, nearly equal-sized blocks $[d] = I_1 \cup I_2 \cup \dots \cup I_n$.
- Let $v \in \{\pm 1\}^n$ be a uniformly chosen ± 1 vector.
- Define the sign map $\text{sign} : \mathbb{R} \rightarrow \{\pm 1\}$ that outputs the sign of the input.
- Checksum function:

$$h(x) := \bigwedge_{i=1}^n \left(\prod_{j \in I_i} \text{sign}(x_j) == v_i \right).$$

- $s = h(x)$.

Black-box undetectability: Idea

Lemma 1. *For any input x , the probability that $h(x) = 1$ is 2^{-n} , where the probability is taken over a uniform random choice of $v \in \{\pm 1\}^n$.*

Lemma 2. *Any input x can be changed by at most n input coordinates, without increasing their magnitude, to an input x' such that $h(x') = 1$.*

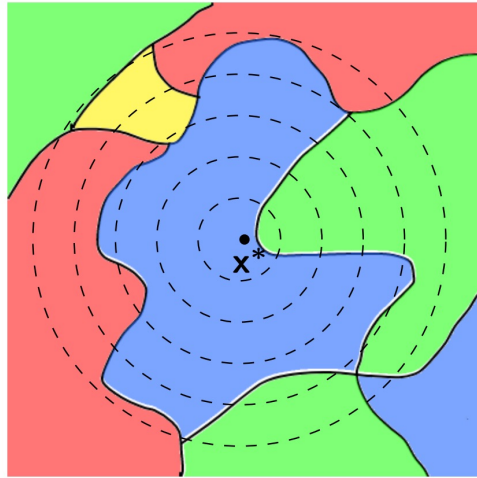
Theorem. *Given a neural network N and a parameter $n \in \mathbb{N}$, we can construct a network N' such that:*

- *For any input x , $N'(x) = N(x)$ with probability $1 - 2^{-n}$.*
- *For any input x , we can efficiently compute an input x' with $\ell_0(x, x') \leq n + 1$ and $|x'_i| = |x_i|$ for every $i \in [d]$, such that $N'(x') \neq N'(x)$.*

Defending against backdoors, without detecting them?



Analogy: a hand sanitizer



- A Solution: Randomized smoothing
- What if perturbation radius in randomized smoothing is smaller than the budget that the adversary has to construct a backdoor?

Program self-correction, via random self-reducibility

- Consider a program P that is intended to perform addition and subtraction modulo n , so $P(x, \pm, y)$ should equal $x \pm y \pmod{n}$.
- Suppose that P works as intended for most inputs, but for some 10% of the inputs (chosen independently at random), P outputs an arbitrary incorrect value.
- Then, instead of using P directly, one could use a program C given by

$$C(x, +, y) = P(P(x, +, u), +, P(y, -, u)),$$

where $u \in \{0, \dots, n-1\}$ is chosen uniformly at random in each invocation of C .

- Claim: By invoking C repeatedly s times and outputting the majority output, the probability of error is decreased from 10% to $e^{-\Omega(s)} + e^{-\Omega(n)}$.

Program self-correction, via random self-reducibility

- P , such that $P(x, \pm, y)$ should equal $x \pm y \pmod{n}$.
- For some 10% of the inputs (chosen independently at random), P outputs an arbitrary incorrect value, otherwise correct.

- Consider

$$C(x, +, y) = P(P(x, +, u), +, P(y, -, u)),$$

where $u \in \{0, \dots, n-1\}$ is chosen uniformly at random in each invocation of C .

- Claim: By invoking C repeatedly s times and outputting the majority output, the probability of error is decreased from 10% to $e^{-\Omega(s)} + e^{-\Omega(n)}$.

Program self-correction, via random self-reducibility

- P , such that $P(x, \pm, y)$ should equal $x \pm y \pmod{n}$.
- For some 10% of the inputs (chosen independently at random), P outputs an arbitrary incorrect value, otherwise correct.
- Consider

$$C(x, +, y) = P(P(x, +, u), +, P(y, -, u)),$$

where $u \in \{0, \dots, n-1\}$ is chosen uniformly at random in each invocation of C .

- Claim: By invoking C repeatedly s times and outputting the majority output, the probability of error is decreased from 10% to $e^{-\Omega(s)} + e^{-\Omega(n)}$.

Notice that in this scheme, we “smooth” using queries very far away from the input.

Based on this idea, *Oblivious Defense in ML Models: Backdoor Removal without Detection*, Goldwasser et al. 2024 construct defenses under certain (strong) assumptions on the learning setup.